Cómo se resolvieron los primeros 5 casos de uso

Sistema de Obras · rama `django-obras`

11 de noviembre de 2026

1 Resumen

La rama `django-obras` del repositorio del equipo tiene 5 casos de uso funcionando: CU-13, CU-01, CU-03 (con CU-06), CU-09 (con CU-10) y CU-04 (con CU-07). Este documento explica cómo se resolvió cada uno: sus pantallas, los archivos donde vive y las reglas que aplica. Al final están las pruebas automáticas que lo comprueban.

CU	Nombre	Quién lo usa	Reglas en
CU-13	Crear cuentas para cada rol	Administrador	`cuentas.py`, `permisos.py`
CU-01	Crear catálogo de conceptos	Superintendente	`views.py`
CU-03/06	Crear y validar reportes diarios	Supervisor crea, Residente valida	`reportes.py`
CU-09/10	Hacer y firmar notas de bitácora	Residente, Supervisor, Superintendente	`bitacora.py`
CU-04/07	Crear y validar reportes mensuales	Supervisor crea, Residente valida	`mensuales.py`

Cómo está armado. Todos los CU siguen el mismo camino:

1. `obras_site/urls.py` recibe la ruta y llama a una vista de `obras/views.py`.
2. La vista revisa el permiso con `@requiere_permiso(...)` de `obras/permisos.py`. Los permisos dependen del rol que el usuario tiene *en esa obra*, no de un rol global.
3. La vista delega las reglas de negocio (estados, quién decide, correos) al módulo del CU.
4. Los datos se guardan con los modelos de `obras/models.py` (`managed = False`): las tablas las crea el SQL de `db/`, no las migraciones de Django.
5. Una plantilla de `obras/templates/` muestra el resultado.

La bandeja `/pendientes/` junta lo que cada usuario tiene por hacer: reportes por revisar o corregir y notas por firmar.

2 CU-13 · Crear cuentas para cada rol

El Administrador crea las cuentas y les asigna un rol *por obra*: una persona puede ser Supervisor en una obra y Residente en otra. Todo cambio queda en un historial.

Ruta	Qué hace
`/usuarios/`	Lista de cuentas con búsqueda por nombre o correo y filtro activas/inactivas
`/usuarios/nuevo/`	Alta: nombre, apellidos, correo, rol, obras y contraseña inicial
`/usuarios/<id>/`	Ficha: editar datos, activar o desactivar, restablecer contraseña, asignar y quitar roles, historial
`/obras/<id>/roles/`	Equipo de una obra: agregar o quitar personas y roles
`/cuenta/password/`	Cada usuario cambia su propia contraseña

Archivos. `obras/cuentas.py`, `obras/views.py` (`usuarios_lista`, `usuario_nuevo`, `usuario_detalle`, `roles_obra`), `obras/forms.py` (`CuentaNuevaForm`, `CuentaEdicionForm`, `CuentaPasswordForm`, `AsignacionFor`

Reglas.

- Administra cuentas quien está en `ADMIN_EMAILS` o tiene el rol Administrador en alguna obra; este último solo asigna roles en las obras que administra.
- Al crear se elige Superintendente, Supervisor, Residente o Directivo. El rol Administrador se da después, desde la ficha.
- El correo no se repite (sin distinguir mayúsculas). La contraseña pasa los validadores de Django y se confirma dos veces.
- Nadie desactiva su propia cuenta ni se quita su propio rol de Administrador. Solo un administrador del sistema modifica a otro.
- Una cuenta desactivada no inicia sesión ni recibe correos.
- Cada alta y cambio se guarda en `django_admin_log` con quién y cuándo.

Permiso	Admin.	Residente	Supervisor	Superint.	Directivo
Consultar la obra	sí	sí	sí	sí	sí
Gestionar usuarios y roles	sí				
Gestionar catálogo	sí			sí	
Crear reportes y programa	sí		sí		
Validar reportes	sí	sí			
Crear y firmar notas de bitácora	sí	sí	sí	sí	

3 CU-01 · Crear catálogo de conceptos

El Superintendente captura el catálogo completo en una sola pantalla; se bloquea en cuanto una estimación usa alguno de sus conceptos.

Ruta	Qué hace
`/obras/<id>/catalogo/`	Ver el catálogo: conceptos por clave, importe total y diferencia contra el contrato
`/obras/<id>/catalogo/editar/`	Capturar o corregir todos los conceptos a la vez

Cada concepto lleva capítulo (opcional), clave, descripción, unidad, cantidad y precio unitario. El importe lo calcula la base de datos en una columna generada.

Archivos. `obras/views.py` (`catalogo_ver`, `catalogo_editar`), `obras/forms.py` (`ConceptoForm`, `BaseConceptoFormSet`), plantillas `catalogo.html` y `catalogo_editar.html`.

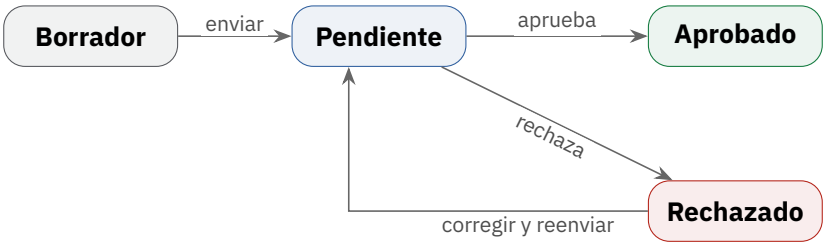
Reglas.

1. **Contrato primero:** sin contrato registrado no deja capturar.
2. **Catálogo en uso:** si una estimación ya usa un concepto, la edición se bloquea.
3. **Clave única** dentro del catálogo, sin distinguir mayúsculas; el error sale en el renglón repetido.
4. **Valores positivos:** cantidad mayor que 0 (4 decimales) y precio mayor que 0 (2 decimales).
5. **Todo o nada:** altas, cambios y bajas en una sola transacción; con claves provisionales se pueden intercambiar claves entre renglones.
6. **Aviso de diferencia:** si el total no coincide con el monto del contrato, avisa pero sí guarda.

La versión de Arale (`/conceptos/`) agregaba un concepto a la vez; se dejó esta porque captura todo junto, compara contra el contrato y bloquea el catálogo en uso. Su commit sigue en el historial.

4 CU-03 y CU-06 · Reportes diarios

El Supervisor elabora un reporte por día y obra; el Residente lo aprueba o lo rechaza una sola vez. El sistema compara el avance real contra el programa de obra.



Estados de un reporte diario o mensual. Solo decide el Residente.

Ruta	Qué hace
`/obras/<id>/reportes/`	Lista de reportes con su estado
`/obras/<id>/reportes/nuevo/`	Nuevo reporte: guardar borrador o enviar a revisión
`/obras/<id>/reportes/<id>/`	Detalle: desviación contra el programa, resumen financiero y validación
`.../editar/ y .../pdf/`	Corregir mientras no esté aprobado y exportar a PDF
`/obras/<id>/programa/`	Programa de obra: fechas con su avance programado en %

Datos. Fecha, inicio y fin de jornada, clima, temperatura, actividades, incidencias, incidentes de seguridad, avance físico real (%) y observaciones.

Archivos. `obras/reportes.py`, `obras/views.py` (`reporte_*`, `programa_*`), `obras/forms.py` (`ReporteDiario`, `ValidacionForm`), `obras/pdf.py`.

Reglas.

- **Un reporte por día y obra,** y no se reporta un día que aún no llega. El fin de jornada debe ser posterior al inicio; el avance va de 0 a 100 %.
- **Borrador privado:** solo lo ven su autor y los administradores, y no avisa a nadie.
- **Solo valida el Residente,** una vez; para rechazar el comentario es obligatorio.
- **Corrección:** un rechazado que se corrige vuelve a **Pendiente** y se reinicia la decisión. Un aprobado ya no se edita.
- **Administrador:** puede cambiar la decisión del Residente, incluso regresarla a **Pendiente**.
- **Desviación:** el avance programado del día se interpola en línea recta entre las fechas del programa (desde 0 % en el inicio oficial). Se muestra «Atraso», «Adelanto» o «En tiempo» en puntos porcentuales.
- **Resumen financiero:** avance físico × monto del contrato contra lo aprobado en estimaciones.
- **Correos:** al enviar, al Residente; con el resultado, al Supervisor.

5 CU-09 y CU-10 · Bitácora de obra

Residente, Supervisor y Superintendente escriben notas con folio consecutivo por obra; la nota queda **Firmada** cuando firman todos los roles presentes.

Ruta	Qué hace
`/obras/<id>/bitacora/`	Lista con búsqueda por estado, tipo, texto o folio
`/obras/<id>/bitacora/nueva/`	Nueva nota: tipo, contenido y nota a la que responde (opcional)
`/obras/<id>/bitacora/<id>/`	Detalle con las firmas de cada rol
`.../editar/`, `.../borrar/`, `.../firmar/`	Corregir, borrar, firmar o rechazar

Tipos de nota. Apertura, Instrucción, Solicitud, Avance, Incidencia, Cierre y Otra (por omisión).

Archivos. `obras/bitacora.py`, `obras/views.py` (`bitacora_lista`, `nota_*`), `obras/forms.py` (`NotaBitacora` `FirmaNotaForm`), `db/cu09_bitacora.sql`.

Reglas.

- **Folio:** lo asigna la base de datos con un contador por obra y nunca se reutiliza, aunque se borre la nota.
- **Quién firma:** los roles Residente, Supervisor y Superintendente con al menos una persona activa en la obra. Administrador y Directivo no firman.
- **El autor firma solo** con cada rol firmante que tenga, y se avisa por correo a los que faltan.
- **Estados:** **Firmada** cuando firman todos; **Rechazada** en cuanto uno rechaza (motivo obligatorio). Cada rol decide una vez.

- **Editar y borrar:** el autor, mientras nadie más haya firmado. Si la rechazan, la corrige y se reinician las firmas.
- **Administrador:** corrige cualquier firma, incluso la regresa a **Pendiente**.
- **La nota relacionada** debe ser de la misma obra.

6 CU-04 y CU-07 · Reportes mensuales

El Supervisor escribe solo el resumen; el sistema arma el resto con los diarios, el programa, las estimaciones y la bitácora. Lo valida el Residente y, al aprobarse, las cifras quedan congeladas.

Ruta	Qué hace
`/obras/<id>/mensuales/`	Lista con su estado
`/obras/<id>/mensuales/nuevo/`	Año, mes, fecha de corte, resumen y observaciones, con vista previa
`/obras/<id>/mensuales/<id>/`	Consolidado, advertencias y validación
`.../editar/`, `.../validar/`, `.../pdf/`	Corregir, validar y exportar a PDF

Qué consolida (del día 1 a la fecha de corte): diarios enviados y aprobados, días sin reporte, avance real al inicio y al corte, avance programado y diferencia, clima, incidencias, incidentes de seguridad, estimaciones aprobadas del mes y notas de bitácora.

Archivos. `obras/mensuales.py`, `obras/views.py` (`mensual_*`), `obras/forms.py` (`ReporteMensualForm`), `db/cu04_reportes_mensuales.sql`.

Reglas.

- **Un reporte por mes y obra.** No se reporta un mes que aún no empieza. La fecha de corte vacía es el último día del mes y siempre cae dentro del mes. El mes no cambia después de creado.
- **Advierte, no bloquea:** días sin diario, diarios sin aprobar, ningún diario aprobado al corte o una obra sin programa.
- **Mismo flujo que el diario:** el Residente aprueba o rechaza una vez.
- **Congelado:** al aprobarse, el consolidado se guarda en JSONB y ya no cambia aunque se modifique un diario; si el Administrador lo regresa a **Pendiente**, se libera.
- **Correos:** al enviar, al Residente con las advertencias; con el resultado, al Supervisor y, si se aprueba, a los Directivos.

7 Pruebas

La rama tiene **35 pruebas automáticas y todas pasan** (`python manage.py test obras`). Corren contra una base PostgreSQL temporal que carga `db/esquema_obras.sql` y los scripts `db/cu*.sql`, así que también prueban los disparadores y restricciones reales.

Caso de uso	Qué se prueba
CU-09/10 Bitácora (18)	Folio consecutivo que no se reutiliza · el autor firma solo y se avisa a los demás · el Administrador no firma al crear · el Directivo no crea ni firma · la nota relacionada debe ser de la misma obra · **Firmada** cuando todos firman · cada rol decide una vez · rechazo con motivo obligatorio · solo firman los roles presentes · el Administrador corrige firmas · editar y borrar antes de otras firmas, no después · corregir y reenviar una rechazada · una firmada no se edita · filtros de la lista · sin acceso a la obra no se ve · el script SQL se puede correr dos veces
CU-04/07 Mensuales (13)	Cifras del consolidado · corte a mitad de mes · congelado al aprobar y liberado al regresar a **Pendiente** · correos de envío y aprobación · el Residente decide una vez · el Superintendente no valida · rechazo, corrección y reenvío · borrador privado y aprobado no editable · permisos por rol · bandeja y PDF · formulario: mes repetido, corte fuera del mes, **mes a futuro**, resumen vacío · vista previa · script SQL idempotente
CU-03/06 Diarios (4)	El Residente aprueba y queda **Aprobado** · el Superintendente recibe «sin permiso» · no se reporta un día futuro · el comando que recalcula estatus

Fechas futuras: ni el reporte diario ni el mensual aceptan un día o un mes que aún no llega. Esta rama no tiene pruebas propias de CU-13 ni de CU-01.

8 Cómo correr el proyecto

```
git checkout django-obras && git pull
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
createdb obra_db
psql -d obra_db -v ON_ERROR_STOP=1 -f db/esquema_obras.sql
psql -d obra_db -f db/cu03_reportes_diarios.sql
psql -d obra_db -f db/cu09_bitacora.sql
psql -d obra_db -f db/cu04_reportes_mensuales.sql
python manage.py migrate && python manage.py createsuperuser
python manage.py runserver
```

Pruebas: `python manage.py check` y `python manage.py test obras` (el usuario de PostgreSQL necesita permiso CREATEDB).